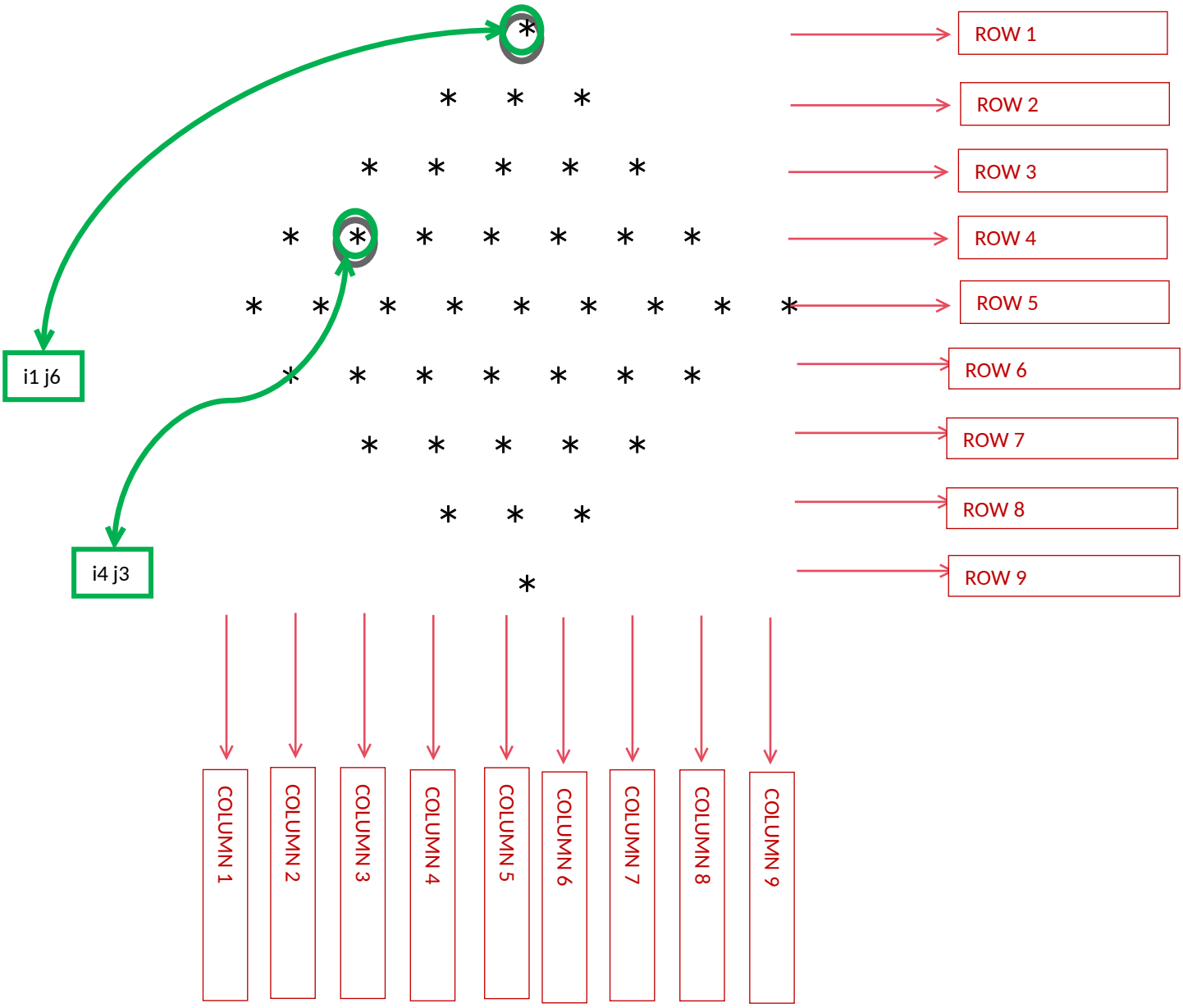
WEEK 01 MODULE
Programming Practice

TOPIC : 1 ' Star Pattern'

1. DIAMOND = (PYRAMID + INVERTED PYRAMID)



1. Problem Analysis :-

i. Pattern Structure:

- The output forms a centered diamond

- Unlike previous patterns, the diamond is not generated using a single growth rule.

Instead, it consists of two separate parts:

Upper Pyramid
+
Lower Inverted Pyramid

- The upper half expands outward from the center.

- The lower half contracts inward toward the center.

- Together, they create a perfectly symmetrical diamond.

ii. Execution Flow:

Step 1: Observe the Grid Concept:
- For n = 5

```
    *
   * * *
  * * * * *
 * * * * * * *
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

Upper Pyramid :-

```
   *
  * * *
 * * * * *
```

```
 * * * * * * *
* * * * * * * * *
```

Lower Pyramid :-

```
 * * * * * * *
   * * * * *
     * * *
       *
```

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

-The pattern can be viewed as two regions:

- **Region 1: Leading Spaces**

These spaces shift the stars toward the center.

- **Region 2: Stars**

These form the visible diamond

For n = 5

Row 0 : [   ][*]

Row 1 : [   ][* * *]

Row 2 : [ ][* * * * *]

Row 3 : [ ][* * * * * * *]

Row 4 : [][* * * * * * * * *]

Row 3 : [ ][* * * * * * *]

Row 2 : [ ][* * * * *]

Row 1 : [   ][* * *]

Row 0 : [   ][*]

UPPER HALF :-

- Spaces decrease by 1.
- Stars increase by 2.

LOWER HALF :-

- Spaces increase by 1.
- Stars decrease by 2.

-> Row Analysis :-

Upper Half :-

Row Index (I)	0	1	2	3	4
Spaces	4	3	2	1	0
Stars	1	3	5	7	9

RELATIONSHIP :-

- Spaces = n - i - 1
- Stars = 2 * i + 1

Lower Half :-

Row Index (I)	3	2	1	0	
Spaces	1	2	3	4	
Stars	7	5	3	1	

RELATIONSHIP :-

- Spaces = n - i - 1
- Stars = 2 * i + 1

The formulas remain exactly the same for both halves.

Only the direction of row traversal changes.

Upper Half:

i = 0 → n - 1

Lower Half:

i = n - 2 → 0

Pyramid (Upper Half):
Spaces ↓
Stars ↑

Inverted Pyramid (Lower Half):
Spaces ↑
Stars ↓

Step 2: Traverse the grid row by row.

- The outer loop is responsible for generating rows.

Generate the upper half using the Pyramid Pattern.

- Traverse rows from 0 to n-1.
- Print the required spaces.
- Print the required stars.

After the upper pyramid is completed, generate the lower half.

- Traverse rows from n-2 down to 0.
- Print the required spaces.
- Print the required stars.

Step 3: After all stars of the current row have been printed, move the cursor to the next line.

- This ensures that the next row starts on a new line.

Step 4: Repeat the process for all rows until the outer loop terminates.

2. Condition Analysis

No conditional statements are required.

There is no need for if(...)

- Because every position visited by the space loop prints a space.

- Every position visited by the star loop prints a star.

- The pattern is controlled entirely through loop limits.

3. Core Logic Transformation

Pyramid Pattern:-

for(int j = 0; j < 2 * i + 1; j++)

- **Meaning:**

Stars increase by 2 in every row.

Additionally :-

for(int j = 0; j < n - i - 1; j++)

- **Meaning:**

controls the alignment of the pattern.

---

for(int i = 0; i < n; i++)

Meaning:

Generates the expanding upper pyramid.

---

for(int i = n - 2; i >= 0; i--)

Meaning:

Generates the shrinking lower pyramid.

- The formulas for spaces and stars remain exactly the same.

- Only the row traversal direction changes.

- This creates the complete diamond shape.

4. Program :-

```
package week1_module;
import java.util.Scanner;
	public class StarDiamond {

		public static void main(String[] args) {
			Scanner sc = new Scanner(System.in);

			System.out.print("Enter the number of rows: ");
			int n = sc.nextInt();
			// Upper Pyramid
			for(int i = 0; i < n; i++) {

				for(int j = 0; j < n - i - 1; j++) {
					System.out.print(" ");
				}

				for(int j = 0; j < 2 * i + 1; j++) {
					System.out.print("* ");
				}

				System.out.println();
			}

			// Lower Inverted Pyramid
			for(int i = n - 2; i >= 0; i--) {

				for(int j = 0; j < n - i - 1; j++) {
					System.out.print(" ");
				}

				for(int j = 0; j < 2 * i + 1; j++) {
					System.out.print("* ");
				}

				System.out.println();
			}

			sc.close();
		}

	}
```

✦ Why n - 2?

Many students write:

for(int i = n - 1; i >= 0; i--)

and get:

```
      *
    * * *
  * * * * *
 * * * * * * *
* * * * * * * * *
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

here the Middle row is printed twice

as Upper Pyramid already printed:

****

So we start from:
i = n - 2

which skips the duplicate middle row.

✦ Upper Part Formulas

Spaces: n - i - 1

Stars: 2*i + 1

✦ Lower Part Formulas

Exactly the same formulas.

The only thing changing is:

i-- instead of i++

which causes the pyramid to shrink.

5. Program Output :-

Enter the number of rows: 5

```
    *
   * * *
  * * * * *
 * * * * * * *
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

6. ScreenShot :-

i. Program :-

StarDiamond.java

```
1  package week1_module;
2  import java.util.Scanner;
3  public class StarDiamond {
4
5  public static void main(String[] args) {
6
7          Scanner sc = new Scanner(System.in);
8
9          System.out.print("Enter the number of rows: ");
10         int n = sc.nextInt();
11      // Upper Pyramid
12         for(int i = 0; i < n; i++) {
13
14             for(int j = 0; j < n - i - 1; j++) {
15                 System.out.print("  ");
16             }
17
18             for(int j = 0; j < 2 * i + 1; j++) {
19                 System.out.print("* ");
20             }
21
22             System.out.println();
23         }
24
25         // Lower Inverted Pyramid
26         for(int i = n - 2; i >= 0; i--) {
27
28             for(int j = 0; j < n - i - 1; j++) {
29                 System.out.print("  ");
30             }
31
32             for(int j = 0; j < 2 * i + 1; j++) {
33                 System.out.print("* ");
34             }
35
36             System.out.println();
37         }
38
39         sc.close();
40     }
41
42 }
43
```

ii. Program's Output :-